Gestion des événements universitaires

Équipe Warriors :
Eya Ben Saber · Eya Baccouche · Rayen Amri · Youssef Saibi · Hana Abidi · Amal Barhoumi

1. Le projet

Une plateforme web unique pour gérer tout le cycle d'un événement universitaire : inscription, réservation de salle, sponsoring, compétitions et équipes, logistique, avis et fidélité. Elle est construite en sept microservices : six individuels, un par membre de l'équipe, et un microservice commun, Logistique & Équipements, développé par toute l'équipe en Python.

Le problème

Les inscriptions sont suivies dans des fichiers dispersés, les salles sont réservées à la main avec un risque de double réservation, le matériel et les sponsors se gèrent par échanges informels, et rien n'aide les étudiants à former une équipe ni ne récompense leur engagement.

Hors périmètre

Paiement en ligne, diffusion vidéo et gestion des comptes, confiée à Keycloak (il n'y a donc pas de microservice utilisateur).

Acteurs et fonctionnalités

Participant (étudiant) — PARTICIPANT

- Consulter les événements et s'inscrire
- Créer ou rejoindre une équipe
- Demander le Smart Matching solo
- Déposer le projet / livrable
- Consulter et convertir ses points de fidélité
- Donner un avis et évaluer

Organisateur (club / BDE) — ORGANISATEUR

- Gérer les événements : créer, publier, modifier
- Réserver une salle ou un amphithéâtre
- Affecter intervenants et jurys
- Emprunter du matériel logistique (kits IoT, micros, câbles)
- Solliciter un sponsor pour un événement

Intervenant / Jury — INTERVENANT

- Déclarer ses disponibilités
- Consulter ses affectations (speaker / jury)
- Évaluer les projets des équipes

Sponsor / Partenaire — SPONSOR

- Consulter les packs de sponsoring
- Traiter une sollicitation (accepter / refuser)
- Souscrire une convention de partenariat

Administrateur — ADMIN

- Gérer le référentiel des salles
- Gérer le stock d'équipements logistiques
- Superviser les événements et conventions

2. Architecture et technologies

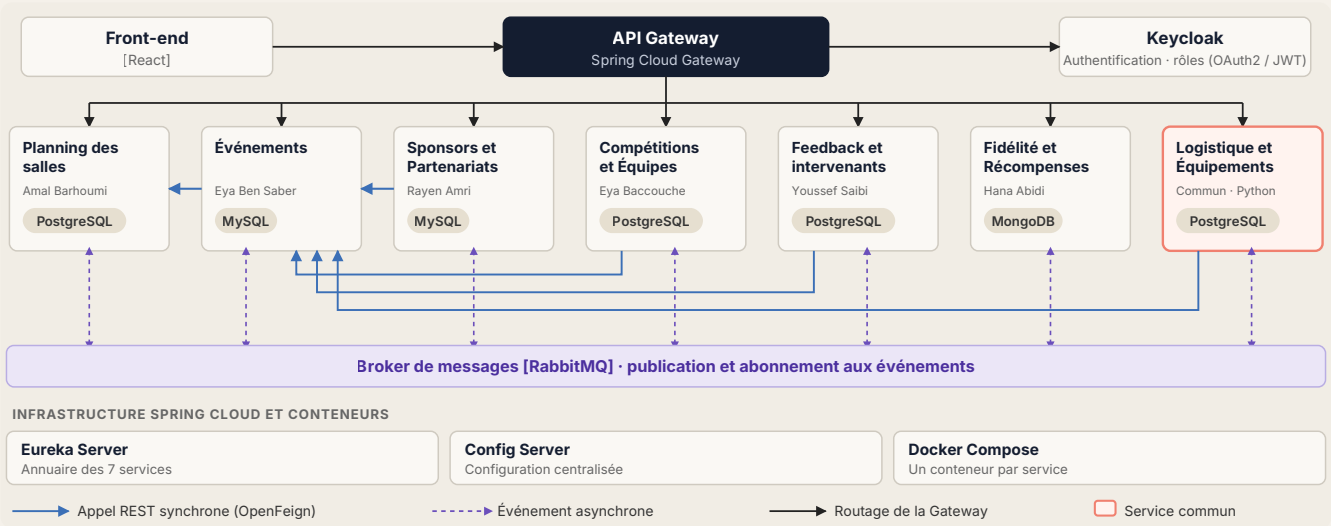
Le front-end n'appelle que l'API Gateway. Elle vérifie les jetons Keycloak puis envoie chaque requête au bon microservice. Chaque service possède sa propre base de données, s'enregistre dans Eureka et récupère sa configuration auprès du Config Server. Le service Python s'enregistre lui aussi dans Eureka, ce qui permet à la Gateway de le router comme les autres.

Composant	Technologie	Rôle	Port
Microservices métier	Spring Boot, Java 17, Maven	Un service par étudiant, documenté avec Swagger	8081 à 8086
Microservice commun	Python	Logistique & Équipements, développé par toute l'équipe	8087
API Gateway	Spring Cloud Gateway	Point d'entrée unique, routage, vérification des jetons JWT	8080
Annuaire	Eureka Server	Enregistrement et découverte des services	8761
Configuration	Config Server	Configuration centralisée	8888
Sécurité	Keycloak	Comptes, rôles et jetons (OAuth2 / JWT)	8180
Messages	RabbitMQ	Événements asynchrones entre services	5672
Persistance	MySQL, PostgreSQL, MongoDB	Une base par service, adaptée à ses données	—
Front-end	React	Application web unique pour le groupe	5173
Conteneurs	Docker et Docker Compose	Un conteneur par service, lancement global	—

Persistance polyglotte

Base	Services	Pourquoi
MySQL	Événements, Sponsors & Partenariats	Transactions et contraintes d'intégrité (capacité, conventions)
PostgreSQL	Planning, Compétitions, Feedback, Logistique	Contraintes d'exclusion, jointures et agrégations SQL
MongoDB	Fidélité & Récompenses	Documents imbriqués et écritures fréquentes

Architecture technique



3. Les sept microservices

MICROSERVICE 1 SUR 7 — EYA BEN SABER

Événements

Crée et publie les événements, gère les inscriptions et contrôle la capacité. C'est le service pivot : tous les appels synchrones passent par lui.

- Suit la vie d'un événement : brouillon, publié, clôturé ou annulé.
- Inscrit les participants tant qu'il reste de la place.
- Prévient les autres services à la création et à chaque inscription.

Données MySQL

Événement : id, titre, description, type, dateDebut, dateFin, salleId, capacité, statut

Inscription : id, evenementId (ManyToOne), participantId, dateInscription, statut

Pourquoi MySQL : données relationnelles et contrôle transactionnel de la capacité.

API REST (extrait)

Méthode	URI	Codes
POST	/events	201 · 400
GET	/events?type=&statut=	200
PUT	/events/{id}/publish	200 · 409
POST	/events/{id}/registrations	201 · 409
GET	/api/evenements/{id}/statut	200 · 404
DELETE	/registrations/{id}	204 · 404

Interactions

SYNC Appelle Planning pour vérifier la disponibilité de la salle avant de publier : un conflit bloque la publication. Appelé par Sponsors, Compétitions, Feedback et Logistique.

PANNE Si Planning est arrêté, l'événement reste en brouillon.

ASYNC Publie *evenement.cree* et *inscription.validee*. Consomme *materiel.restitue* (clôture du volet matériel).

MICROSERVICE 2 SUR 7 — EYA BACCOUCHE

Compétitions & Équipes

Organise les hackathons : équipes, dépôt de projets et Smart Matching pour les étudiants solos sans équipe.

- Crée les équipes ou fait rejoindre par code d'invitation.
- Propose une équipe aux étudiants solos (Smart Matching).
- Reçoit le dépôt du projet ; le jury attribue la note finale.

Données PostgreSQL

Equipe : id, evenementId, nomEquipe, codeInvitation, tableId, dateCreation

MembreEquipe : id, equipeId, participantId, roleCompetence, estChefEquipe

ProjetSoumis : id, equipeId, titre, repoUrl, dateDepot, noteFinale

Pourquoi PostgreSQL : requêtes relationnelles (équipes, membres, projets) et calcul du Smart Matching en SQL.

API REST (extrait)

Méthode	URI	Codes
POST	/competitions/teams	201 · 403 · 409
POST	/competitions/teams/join	200 · 404 · 409
POST	/competitions/matching	200 · 404
POST	/competitions/teams/{id}/projects	201 · 409
PUT	/competitions/projects/{id}/grade	200 · 403 · 404

Interactions

SYNC Appelle Événements pour vérifier que l'étudiant est inscrit avant d'intégrer une équipe (*GET /api/evenements/{id}/inscriptions/{etudiantId}*).

PANNE Si Événements est arrêté, la création ou l'adhésion est refusée, puis nouvel essai.

ASYNC Publie *equipe.formee*. Ne consomme aucun message.

MICROSERVICE 3 SUR 7

RAYEN AMRI

Sponsors & Partenariats

Gère les packs de sponsoring, les conventions et le démarchage entre organisateurs et sponsors.

- Définit les packs de sponsoring : tarif, stand inclus.
- L'organisateur sollicite un sponsor ; il accepte ou refuse.
- Une sollicitation acceptée donne une convention de partenariat.

Données MySQL

Sponsor : id, nomEntreprise, contactEmail, logoUrl
PackSponsoring : id, nomPack, tarif, standInclus
ConventionPartenariat : id, sponsorId, packId, evenementId, budgetAlloue, statutPaiement

Pourquoi MySQL : contraintes d'intégrité et transactions sur les conventions.

API REST (extrait)

Méthode	URI	Codes
POST	/sponsors/packs	201 · 403
GET	/sponsors/packs	200
POST	/sponsors/solicitations	201 · 404 · 409
PUT	/sponsors/solicitations/{id}/accept	200 · 404 · 409
PUT	/sponsors/solicitations/{id}/reject	200 · 404 · 409
POST	/sponsors/conventions	201 · 409

Interactions

SYNC Appelle Événements : l'événement doit exister et être au statut PUBLIÉ (*GET /api/evenements/{id}*).

PANNE Si Événements est arrêté, la sollicitation est refusée, puis nouvel essai.

ASYNC Publie *convention.validee*. Consomme *materiel.restitue* (libération des cautions).

MICROSERVICE 4 SUR 7

YOUSSEF SAIBI

Feedback et intervenants

Recueille les avis des participants et gère les intervenants, jurys et leurs affectations.

- Recueille les avis des participants après l'événement.
- Gère les profils des intervenants et des jurys.
- Les affecte à un événement comme conférencier ou jury.

Données PostgreSQL

Avis : id, evenementId, participantId, note, commentaire, dateAvis
Intervenant : id, nom, fonction, disponibilites
Affectation : id, intervenantId, evenementId, rôle

Pourquoi PostgreSQL : base relationnelle robuste, commune avec Planning, Compétitions et Logistique.

API REST (extrait)

Méthode	URI	Codes
POST	/events/{id}/feedback	201 · 403 · 409
GET	/events/{id}/feedback	200
POST	/speakers	201
POST	/assignments	201 · 409
GET	/speakers/{id}/assignments	200

Interactions

SYNC Appelle Événements : l'événement doit être CLÔTURÉ avant le dépôt d'un avis (*GET /api/evenements/{id}/statut*).

PANNE Si Événements est arrêté, l'avis est refusé et peut être réessayé.

ASYNC Publie *avis.publie*. Ne consomme aucun message.

MICROSERVICE 5 SUR 7

HANA ABIDI

Fidélité & Récompenses

Anime la gamification : points d'engagement, badges et récompenses. Il est uniquement abonné aux événements des autres services.

- Crédite des points d'engagement à chaque action.
- Attribue des badges et un niveau de fidélité.
- Échange les points contre des récompenses.

Données MongoDB

CompteFidelite : id, participantId, soldePoints, niveau, badges, dateDerniereActivite
TransactionPoints : id, participantId, evenementId, typeAction, points, date
Recompense : id, titre, coutPoints, stockRestant, actif

Pourquoi MongoDB : documents imbriqués (historique de points) et écritures fréquentes.

API REST (extrait)

Méthode	URI	Codes
GET	/loyalty/account/me	200
GET	/loyalty/transactions/me	200
GET	/loyalty/rewards	200
POST	/loyalty/rewards	201 · 403
POST	/loyalty/rewards/{id}/redeem	200 · 409

Interactions

SYNC Aucun appel vers les autres services.

ASYNC Consomme *inscription.validee* (+50 points), *equipe.formee* (badge Team Player, +100 points) et *avis.publie* (+20 points).

PANNE Si Fidélité est arrêté, les messages attendent dans le broker et les points sont crédités à son retour.

MICROSERVICE 6 SUR 7 AMAL BARHOUMI

Planning des salles

Gère les salles et empêche les réservations qui se chevauchent.

- Gère les salles et leurs réservations.
- Refuse tout chevauchement de créneau.
- Verrouille la salle dès la création de l'événement.

Données PostgreSQL

Salle : id, nom, batiment, capacite, typeSalle

Reservation : id, salleId, evenementId, debut, fin, statut

Pourquoi PostgreSQL : une contrainte d'exclusion sur les plages horaires interdit les conflits.

API REST (extrait)

Méthode	URI	Codes
GET	/rooms?minCapacity=	200
POST	/rooms	201 · 403
POST	/reservations	201 · 409 · 422
DELETE	/reservations/{id}	204 · 404
GET	/api/salles/{id}/disponibilite	200 · 404

Interactions

SYNC Appelé uniquement par Événements (disponibilité de la salle sur un intervalle). Planning n'appelle jamais Événements.

ASYNC Consomme *evenement.cree* : pose un verrou sur la salle retenue et bloque les créneaux concurrents.

MICROSERVICE COMMUN · TOUTE L'ÉQUIPE · PYTHON PORT 8087

Logistique & Équipements

Gère le matériel, les kits IoT et les stands. Développé en commun, en Python.

- Tient l'inventaire du matériel et des kits IoT.
- Traite les demandes d'emprunt des organisateurs.
- Suit les retours et l'état du matériel.

Données PostgreSQL

Equipement : id, designation, categorie, quantiteStock, quantiteDisponible

DemandeEmprunt : id, equipementId, evenementId, demandeurId, quantite, dateEmprunt, dateRetourPrevue, statut

Pourquoi PostgreSQL : transactions et contraintes sur les stocks, comme les autres services relationnels.

API REST (extrait)

Méthode	URI	Codes
GET	/logistics/equipment?category=	200
POST	/logistics/equipment	201 · 403
POST	/logistics/loans	201 · 404 · 409
PUT	/logistics/loans/{id}/return	200 · 404
GET	/logistics/loans?eventId=	200

Interactions

SYNC Appelle Événements : la demande doit être rattachée à un événement vérifié (*GET /api/evenements/{id}*).

PANNE Si Événements est arrêté, la demande d'emprunt est refusée, puis nouvel essai.

ASYNC Consomme *evenement.cree* (pack matériel standard), *equipe.formee* (kits IoT) et *convention.validee* (stand sponsor). Publie *materiel.restitue*.

4. Interactions entre les services

Règle : synchrone quand l'appelant a besoin de la réponse tout de suite (OpenFeign), asynchrone quand il informe les autres sans les attendre (RabbitMQ, exchange topic *evenements.topic*). Il y a cinq appels synchrones, tous vers ou depuis Événements, et six événements asynchrones.

Communications synchrones (Spring Cloud OpenFeign)

Appel	Requête	Règle bloquante et panne
Événements → Planning	GET /api/salles/{id}/disponibilite?debut=&fin=	Salle libre sur l'intervalle, sinon la publication est bloquée. Si Planning est arrêté, l'événement reste en brouillon.
Sponsors → Événements	GET /api/evenements/{evenementId}	L'événement existe et est au statut PUBLIÉ avant une convention. Refus, puis nouvel essai.
Compétitions → Événements	GET /api/evenements/{evenementId}/ inscriptions/{etudiantId}	L'étudiant est inscrit avant d'intégrer une équipe. Refus, puis nouvel essai.
Feedback → Événements	GET /api/evenements/{evenementId}/statut	L'événement est CLÔTURÉ avant le dépôt d'un avis. Refus, puis nouvel essai.
Logistique → Événements	GET /api/evenements/{evenementId}	Le matériel est rattaché à un événement vérifié. Refus, puis nouvel essai.

Communications asynchrones (RabbitMQ)

Clé de routage	Producteur → consommateurs	Traitement
evenement.cree	Événements → Planning, Logistique	Planning verrouille la salle et bloque les créneaux concurrents ; Logistique pré-réserve le pack matériel standard.
inscription.validee	Événements → Fidélité	Fidélité ajoute +50 points au compte du participant, sans bloquer l'inscription.
equipe.formee	Compétitions → Fidélité, Logistique	Fidélité attribue le badge Team Player et +100 points ; Logistique prépare les kits IoT et multiprises.
avis.publie	Feedback → Fidélité	Fidélité crédite +20 points pour l'avis donné.
convention.validee	Sponsors → Logistique	Logistique attribue le stand, les roll-ups et les écrans dès la validation de la convention.
materiel.restitue	Logistique → Événements, Sponsors	Événements clôture le volet matériel ; Sponsors libère les cautions et conventions.

5. Socle commun d’équipe

En plus des six microservices individuels, toute l’équipe développe le microservice commun Logistique & Équipements (Python, port 8087) et le socle d’infrastructure basé sur Spring Cloud et Docker.

Composants communs

Composant	Rôle
Logistique & Équipements (8087)	Microservice Python, PostgreSQL : matériel, kits IoT et stands.
API Gateway (8080)	Point d’entrée unique du front-end React (5173) et vérification des jetons JWT Keycloak.
Keycloak (8180)	Comptes, rôles et jetons OAuth2 / JWT ; aucun microservice utilisateur.
Eureka Server (8761)	Annuaire dynamique des 7 microservices.
Config Server (8888)	Centralisation des configurations.
Docker Compose	Orchestration des conteneurs : 7 microservices, bases de données, RabbitMQ, Keycloak et socle Spring Cloud.

Routage API Gateway

Préfixe URI	Service cible	Route Eureka
/events/**	Événements	lb://evenements
/rooms/, /reservations/****	Planning	lb://planning
/feedback/, /speakers/****	Feedback	lb://feedback
/competitions/**	Compétitions & Équipes	lb://competitions
/sponsors/**	Sponsors & Partenariats	lb://sponsors
/loyalty/**	Fidélité & Récompenses	lb://fidelite
/logistics/**	Logistique & Équipements (Python)	lb://logistique